

DAY 2 — CONDITIONS

Python Programming Bootcamp

Topics: if · if-else · nested conditions · logical & comparison operators

? Today's Agenda

01 Comparison Operators

02 Logical Operators

03 if Statement

04 if-else Statement

05 Nested Conditions

06 Industry Mini Project: Loan Eligibility

? What are Conditions?

A condition is a question your program asks — and based on the answer (True/False), it decides what to do next.

? Real-World Analogy

If it is raining → carry an umbrella

Else → enjoy the sunshine ☀

```
# Python mirrors realworld decisions
if raining:
    carry_umbrella()
else:
    enjoy_sunshine()
```

Key Concepts Today

- ? Boolean values: True / False
- ? Comparison operators
- ? Logical operators
- ? if, if-else, nested if

Conditions are the backbone of decision-making in every program!



Comparison Operators

Compare two values — result is always True or False

Comparison Operators

Op	Meaning	Example	Result
<code>==</code>	Equal to	<code>5 == 5</code>	True
<code>!=</code>	Not equal to	<code>5 != 3</code>	True
<code>></code>	Greater than	<code>7 > 4</code>	True
<code><</code>	Less than	<code>2 < 8</code>	True
<code>>=</code>	Greater than or equal	<code>5 >= 5</code>	True
<code><=</code>	Less than or equal	<code>3 <= 7</code>	True

 `==` checks equality • `=` assigns a value. Don't confuse them!

⚖ Comparison Operators — Code Examples

```
age = 20
print(age >= 18) # True
print(age == 20) # True
print(age != 20) # False
print(age < 25)  # True
print(age > 30)  # False
```

```
salary = 50000
m_in_salary = 30000

print(salary > m_in_salary) # True
print(salary == m_in_salary) # False
print(salary <= 60000)     # True
```

🔑 Key Points

- Comparison operators always return a Boolean: True or False
- They work on integers, floats, and strings
- Strings are compared alphabetically ('a' < 'b' is True)
- Result can be stored: result = (age >= 18)



Logical Operators

Combine multiple conditions with and, or, not

? Logical Operators

and

True only if BOTH conditions are True

True and True → True

True and False → False

or

True if AT LEAST ONE condition is True

True or False → True

False or False → False

not

REVERSES the Boolean value

not True → False

not False → True

```
age = 22  
salary = 45000
```

```
# and  
if age >= 18 and salary >= 30000:  
    print("Eligible") # Eligible
```

```
# or  
if age < 18 or salary < 20000:  
    print("Not qualified")
```



if Statement

Execute code only when a condition is True

? The if Statement

Syntax

```
if condition:  
    # code to execute
```

Rules

- Colon (:) is mandatory after condition
- Indentation (4 spaces) defines the block

```
# Example 1: Age check  
age = 20  
if age >= 18:  
    print("You are an adult")  
    print("You can vote!")  
  
# Example 2: Positive number  
num = 15  
if num > 0:  
    print("Positive number")
```

⚠ Common Mistake:

`if age = 18:` ← WRONG (assignment, not comparison) | `if age == 18:` ← CORRECT



if-else Statement

Handle both True and False cases

? The if-else Statement

Syntax

```
if condition:  
    # runs when True  
else:  
    # runs when False
```

Flow

Condition True → run if-block
Condition False → run else-block
Always exactly ONE block runs!

```
# Exam ple 1: Pass or Fail
```

```
marks = 35
```

```
if marks >= 40:
```

```
    print("PASS")
```

```
else:
```

```
    print("FAIL")
```

```
# Exam ple 2: Even or Odd
```

```
number = 7
```

```
if number % 2 == 0:
```

```
    print(f"{number} is Even")
```

```
else:
```

```
    print(f"{number} is Odd") # Output: 7 is Odd
```

```
# Exam ple 3: Voting eligibility
```

```
age = 16
```

? Every if can have an else — it covers ALL remaining cases.

? elif — Multiple Conditions

```
# Grade calculator using elif
marks = 75

if marks >= 90:
    grade = "A+"
elif marks >= 75:
    grade = "A" # ← This block runs
elif marks >= 60:
    grade = "B"
elif marks >= 40:
    grade = "C"
else:
    grade = "F"

print(f"Grade: {grade}") # Grade: A
```

? elif Rules

- Stands for 'else if'
- Can have multiple elif blocks
- Checked top to bottom
- First matching block runs
- Once matched, rest are skipped
- else is optional at the end

Structure: if → elif → elif → ... → else



Nested Conditions

if inside if — layers of decision-making

? Nested if Statements

A nested if is an if statement inside another if block. Useful for layered conditions.

```
age = 22
income = 50000
employment = "salaried"

if age >= 18:           # Outer condition
    print("Age: Eligible")
    if income >= 30000:  # Inner condition 1
        print("Income: Eligible")
        if employment == "salaried": # Inner condition 2
            print("Loan Approved")
        else:
            print("Employment type not salaried")
    else:
        print("Income too low")
```

⚠ Indentation is Critical!

Each nested level adds 4 spaces.
Wrong indentation = wrong logic or error.

? When to use Nested if?

When conditions depend on each other;
like eligibility checks, game logic.

? Nested if vs Logical Operators

? Nested if

```
if age >= 18:  
    if income >= 30000:  
        print("Eligible")
```

- More readable for complex logic
- Can add extra actions between layers
- Easier to handle each case separately
- Better when conditions have a hierarchy

? Logical Operators

```
if age >= 18 and income >= 30000:  
    print("Eligible")
```

- More concise, fewer lines
- Best when all conditions must be met
- Less readable for complex branching
- No intermediate messages between checks

? Use logical operators for simple combined checks • Use nested if for layered, hierarchical decisions

? Truthiness — What's True and False?

? Falsy Values

0 (zero)

"" (empty string)

None

[] (empty list)

False

0.0 (zero float)

? Truthy Values

Any non-zero number

Non-empty string

Non-empty list

True

Any object

```
name = ""  
if name:  
    print("Name exists")  
else:
```

```
    print("Name is empty") # This runs
```

⚡ Short-Circuit Evaluation

Python stops evaluating as soon as the result is known — saving computation.

and — Short-Circuits on False

If the FIRST condition is False, Python skips the second entirely.

False and <anything> → False (stops here)

or — Short-Circuits on True

If the FIRST condition is True, Python skips the second entirely.

True or <anything> → True (stops here)

```
# Short-circuit with 'and'
age = 15
if age >= 18 and salary >= 30000:
    # salary is never checked because age fails!
    print("Eligible")
```

```
# Practical use: Avoid errors
my_list = []
```

🔍 Short-circuit makes your code efficient and prevents runtime errors!

? Operator Precedence in Conditions

When mixing operators, Python follows this order:

1st	()	Parentheses
2nd	Comparison: ==, !=, <, >, <=, >=	Evaluated first
3rd	not	Logical NOT
4th	and	Logical AND
5th	or	Logical OR (lowest)

```
# Without parentheses – might be confusing
result = True or False and False # True (and binds tighter)
```

```
# With parentheses – clear and explicit
result = (True or False) and False # False
```

Best practice: always use parentheses for clarity!

? When in doubt — use parentheses to make your intent crystal clear.



Industry Mini Project

Loan Eligibility System

? Loan Eligibility System — Requirements

Build a system that decides if a person is eligible for a bank loan based on three factors:

? Age

Must be 21 – 60 years

? Salary

Minimum ₹25,000/month

? Employment Type

Must be 'salaried' or 'self-employed'

```
# Input
age = int(input("Enter age: "))
salary = int(input("Enter monthly salary: "))
employment = input("Employment type (salaried/self-employed/unemployed): ")
```

? ALL three conditions must pass for loan approval | Specific rejection message for each failure

? Loan Eligibility — Full Solution

```
# Loan Eligibility System
age = int(input("Enter age: "))
salary = int(input("Enter monthly salary (₹): "))
employment = input("Employment type: ").lower()

if age < 21 or age > 60:
    print("❌ Rejected: Age must be between 21 and 60")
elif salary < 25000:
    print("❌ Rejected: Minimum salary required is ₹25,000")
elif employment not in ["salaried", "self-employed"]:
    print("❌ Rejected: Employment type not eligible")
else:
    print("✅ Congratulations! Loan Approved!")
```

Sample Outputs

In: age=25, salary=40000, salaried → ✅ Congratulations! Loan Approved!

In: age=17, salary=50000, salaried → ❌ Rejected: Age must be between 21 and 60

In: age=30, salary=20000, salaried → ❌ Rejected: Minimum salary ₹25,000

? Best Practices for Conditions

1

Use parentheses for clarity

`if (age >= 18) and (salary >= 30000):`

2

Avoid deep nesting — use elif instead

Use elif chain instead of if inside if inside if

3

Check most likely condition first

Put the most common case at the top for speed

4

Use 'in' for membership checks

`if employment in ['salaried', 'self-employed']:`

5

Keep conditions readable — one per line if complex

`if (age >= 18 and\n salary >= 30000 and\n employment == 'salaried'):`

? Common Errors & How to Fix Them

? Wrong

```
if age = 18:
```

? Correct

```
if age == 18:
```

Why?

= is assignment, == is comparison

? Wrong

```
if age >= 18 :
```

? Correct

```
if age >= 18:
```

Why?

No space before the colon

? Wrong

```
if age >= 18  
    print('ok')
```

? Correct

```
if age >= 18:  
    print('ok')
```

Why?

Missing colon after condition

? Wrong

```
if age >= 18:  
    print('ok')
```

? Correct

```
if age >= 18:  
    print('ok')
```

Why?

Missing indentation (4 spaces)

? Wrong

```
if age >= 18 AND salary > 0:
```

? Correct

```
if age >= 18 and salary > 0:
```

Why?

Logical operators must be lowercase

Day 2 Summary

Comparison Operators

`== != > < >= <=`

Logical Operators

`and · or · not`

if Statement

Execute code when True

if-else / elif

Handle True and False paths

Nested Conditions

if inside if for layered decisions

Mini Project

Loan Eligibility System

Next: Day 3 — Loops (*for · while · break · continue*)

Comparison Operator

What is the output of the following code?

```
x = 10  
y = 10  
print(x == y)
```

A 10

B True

C False

D Error



Answer 1 — Comparison Operator

```
x = 10  
y = 10  
print(x == y)
```

☐ A) 10

☒ B) True

☐ C) False

☐ D) Error

? Explanation

The == operator checks if two values are equal. Since both x and y are 10, the result is True. Remember: == is comparison, not assignment (=).

? Why Other Options Are Wrong

A) 10 — Wrong! == does not return the value, it returns a Boolean.

C) False — Wrong! x and y are equal (both 10), so the result is True.

D) Error — Wrong! This is valid Python syntax.

Comparison Operator

Which comparison operator checks 'NOT equal to'?

```
a = 5  
b = 3  
print(a ___ b) # Should print True
```

A ==

B <>

C !=

D =/=



Answer 2 — Comparison Operator

```
a = 5  
b = 3  
print(a __ b) # Should print True
```

☐ A) ==

☐ B) <>

☒ C) !=

☐ D) /=

? Explanation

!= is Python's 'not equal to' operator. It returns True if the two values are different. In this case $5 \neq 3 \rightarrow \text{True}$. Note: <> was used in Python 2 but is removed in Python 3.

? Why Other Options Are Wrong

- A) == checks EQUALITY, not inequality.
- B) <> was Python 2 only. Not valid in Python 3.
- D) /= is not a valid Python operator.

if Statement

What will this code print?

```
score = 50  
if score > 100:  
    print("Bonus!")  
print("Done")
```

A Bonus!Done

B Done

C Bonus!

D Nothing



Answer 3 — if Statement

```
score = 50
if score > 100:
    print("Bonus!")
print("Done")
```

☐ A) Bonus!\nDone

☒ B) Done

☐ C) Bonus!

☐ D) Nothing

? Explanation

score (50) is NOT greater than 100, so the if block is skipped. But print('Done') is OUTSIDE the if block (no indentation), so it always runs. Output: Done

? Why Other Options Are Wrong

- A) Bonus!\nDone — Wrong! $50 > 100$ is False, so 'Bonus!' is never printed.
- C) Bonus! — Wrong! The condition is False. Only 'Done' prints.
- D) Nothing — Wrong! print('Done') is outside the if block and always executes.

if-else Statement

What will this code print?

```
num = 0
if num :
    print("Non-zero")
else:
    print("Zero")
```

A Non-zero

B Zero

C 0

D Error



Answer 4 — if-else Statement

```
num = 0
if num :
    print("Non-zero")
else:
    print("Zero")
```

☐ A) Non-zero

☒ B) Zero

☐ C) 0

☐ D) Error

? Explanation

In Python, 0 is Falsy — it evaluates to False in a Boolean context. So the if block is skipped and the else block runs, printing 'Zero'. This is called Truthiness.

? Why Other Options Are Wrong

A) Non-zero — Wrong! 0 is falsy in Python, so the if block doesn't run.

C) 0 — Wrong! print(num) isn't in the code; it prints the string 'Zero'.

D) Error — Wrong! This is valid Python using truthiness evaluation.

Logical Operator: and

What will this code output?

```
age = 20
salary = 40000
if age >= 18 and salary >= 30000:
    print("Eligible")
else:
    print("Not eligible")
```

A Not eligible

B Eligible

C True

D Error



Answer 5 — Logical Operator: and

```
age = 20
salary = 40000
if age >= 18 and salary >= 30000:
    print("Eligible")
else:
    print("Not eligible")
```

☐ A) Not eligible

☒ B) Eligible

☐ C) True

☐ D) Error

? Explanation

Both conditions are True: age (20) >= 18 is True, and salary (40000) >= 30000 is True. With 'and', both must be True. So the if block runs and prints 'Eligible'.

? Why Other Options Are Wrong

A) Not eligible — Wrong! Both conditions pass, so 'Eligible' is printed.

C) True — Wrong! The code prints the string 'Eligible', not a boolean.

D) Error — Wrong! This is valid Python syntax.

Logical Operator: or

What is the output?

```
x = 5
if x < 0 or x > 3:
    print("Out of range")
else:
    print("In range")
```

A In range

B Out of range

C True

D No output



Answer 6 — Logical Operator: or

```
x = 5
if x < 0 or x > 3:
    print("Out of range")
else:
    print("In range")
```

☐ A) In range

☒ B) Out of range

☐ C) True

☐ D) No output

? Explanation

$x = 5$. Check: $5 < 0 \rightarrow \text{False}$. Check: $5 > 3 \rightarrow \text{True}$. With 'or', only ONE needs to be True. Since $5 > 3$ is True, the if block runs: 'Out of range'.

? Why Other Options Are Wrong

A) In range — Wrong! $x > 3$ is True, so the or condition is True.

C) True — Wrong! The code prints the string, not a boolean.

D) No output — Wrong! The if condition is True, so 'Out of range' prints.

Logical Operator: not

What will this print?

```
logged_in = False
if not logged_in:
    print("Please login")
else:
    print("Welcome!")
```

A Welcome!

B Please login

C False

D Error



Answer 7 — Logical Operator: not

```
logged_in = False
if not logged_in:
    print("Please login")
else:
    print("Welcome!")
```

☐ A) Welcome!

☒ B) Please login

☐ C) False

☐ D) Error

? Explanation

logged_in = False. 'not False' = True. So the if block runs and prints 'Please login'. The 'not' operator reverses the Boolean value.

? Why Other Options Are Wrong

A) Welcome! — Wrong! 'not False' is True, so the if block (not else) runs.

C) False — Wrong! The code prints a string, not a boolean value.

D) Error — Wrong! This is perfectly valid Python.

elif Chain

What grade will be printed?

```
m arks = 72
if m arks >= 90:
    print("A+")
elif m arks >= 75:
    print("A")
elif m arks >= 60:
```

A A+

B A

C B

D C



Answer 8 — elif Chain

```
marks = 72
if marks >= 90:
    print("A+")
elif marks >= 75:
    print("A")
elif marks >= 60:
    print("B")
else:
    print("C")
```

☐ A) A+

☐ B) A

☒ C) B

☐ D) C

? Explanation

marks = 72. Check $72 \geq 90 \rightarrow \text{False}$ (skip). Check $72 \geq 75 \rightarrow \text{False}$ (skip). Check $72 \geq 60 \rightarrow \text{True}$! So 'B' is printed. Once a condition matches, the rest are skipped.

? Why Other Options Are Wrong

A) A+ — Wrong! $72 < 90$, so A+ is not printed.

B) A — Wrong! $72 < 75$, so A is not printed.

D) C — Wrong! $72 \geq 60$ is True, so B is printed before reaching else.

Nested if

What is the output?

```
x = 10
if x > 5:
    if x > 8:
        print("Big")
    else:
        print("Medium")
```

A Small

B Medium

C Big

D No output



Answer 9 — Nested if

```
x = 10
if x > 5:
    if x > 8:
        print("Big")
    else:
        print("Medium ")
else:
    print("Small")
```

☐ A) Small

☐ B) Medium

☒ C) Big

☐ D) No output

? Explanation

$x = 10$. Outer if: $10 > 5 \rightarrow \text{True}$, enter outer block. Inner if: $10 > 8 \rightarrow \text{True}$, print 'Big'. The nested structure checks conditions at each level.

? Why Other Options Are Wrong

A) Small — Wrong! $10 > 5$ is True, so the outer else never runs.

B) Medium — Wrong! $10 > 8$ is also True, so the inner if runs, not the inner else.

D) No output — Wrong! Both conditions are True, 'Big' is printed.

Truthiness

Which value is Falsy in Python?

```
# Which of these evaluates to False  
# when used in an if condition?  
if ???:  
    print("This won't print")
```

A "hello"

B [1,2,3]

C []

D 42



Answer 10 — Truthiness

```
# Which of these evaluates to False  
# when used in an if condition?  
if ???:  
    print("This won't print")
```

☐ A) "hello"

☐ B) [1,2,3]

☒ C) []

☐ D) 42

? Explanation

An empty list [] is Falsy in Python. Other falsy values: 0, 0.0, "", None, False. Non-empty strings, lists with items, and non-zero numbers are all Truthy.

? Why Other Options Are Wrong

A) "hello" — Wrong! A non-empty string is Truthy.

B) [1, 2, 3] — Wrong! A non-empty list is Truthy.

D) 42 — Wrong! Any non-zero number is Truthy.

Comparison + Logical

What does this print?

```
a = 5
b = 10
c = 15
if a < b and b < c:
    print("Ascending")
else:
```

A Not ascending

B True

C Ascending

D Error



Answer 11 — Comparison + Logical

```
a = 5
b = 10
c = 15
if a < b and b < c:
    print("Ascending")
else:
    print("Notascending")
```

☐ A) Notascending

☐ B) True

☒ C) Ascending

☐ D) Error

? Explanation

$a < b \rightarrow 5 < 10 \rightarrow \text{True}$. $b < c \rightarrow 10 < 15 \rightarrow \text{True}$. $\text{True and True} \rightarrow \text{True}$. So 'Ascending' is printed. This pattern is used to check ordered sequences.

? Why Other Options Are Wrong

A) Not ascending — Wrong! Both comparisons are True, so 'Ascending' prints.

B) True — Wrong! The code prints a string, not a boolean value.

D) Error — Wrong! This is valid Python syntax.

Short-Circuit

What happens when this runs?

```
x = 0
if x != 0 and 10 / x > 2:
    print("Yes")
else:
    print("No")
```

A ZeroDivisionError

B Yes

C No

D Nothing



Answer 12 — Short-Circuit

```
x = 0
if x != 0 and 10 / x > 2:
    print("Yes")
else:
    print("No")
```

☐ A) ZeroDivisionError

☐ B) Yes

☒ C) No

☐ D) Nothing

? Explanation

Short-circuit evaluation! $x \neq 0 \rightarrow 0 \neq 0 \rightarrow \text{False}$. Since the first part of 'and' is False, Python NEVER evaluates '10 / x'. This prevents a ZeroDivisionError. Prints: No

? Why Other Options Are Wrong

A) ZeroDivisionError — Wrong! Short-circuit saves us: 10/x is never evaluated.

B) Yes — Wrong! $x \neq 0$ is False, so the if block doesn't run.

D) Nothing — Wrong! The else block runs and prints 'No'.

Indentation

What is wrong with this code?

```
age = 20  
if age >= 18:  
print("Adult")
```

A Missing colon

B IndentationError

C Wrong operator

D Nothing wrong



Answer 13 — Indentation

```
age = 20  
if age >= 18:  
print("Adult")
```

☐ A) Missing colon

☒ B) IndentationError

☐ C) Wrong operator

☐ D) Nothing wrong

? Explanation

The print statement is NOT indented under the if. Python uses indentation to define code blocks. Without 4 spaces (or 1 tab), Python raises an IndentationError. Always indent code inside if blocks!

? Why Other Options Are Wrong

- A) Missing colon — Wrong! The colon is present after age >= 18.
- C) Wrong operator — Wrong! >= is the correct 'greater than or equal to' operator.
- D) Nothing wrong — Wrong! The code will crash with IndentationError.

Operator Precedence

What does this evaluate to?

```
result = True or False and False  
print(result)
```

A False

B True

C Error

D None



Answer 14 — Operator Precedence

```
result = True or False and False  
print(result)
```

☐ A) False

☒ B) True

☐ C) Error

☐ D) None

? Explanation

'and' has higher precedence than 'or'. So: False and False → False first. Then: True or False → True. Result: True. Use parentheses to avoid confusion: (True or False) and False → False.

? Why Other Options Are Wrong

A) False — Wrong! 'and' binds tighter, so we get True or (False and False) = True or False = True.

C) Error — Wrong! This is valid Python. Operator precedence resolves the expression.

D) None — Wrong! This expression returns True, not None.

Loan Eligibility

For the Loan Eligibility System, which input combination gets approved?

```
# Conditions:  
# age:21-60  
# salary >= 25000  
# employment:salaried or self-employed
```

A age=19, salary=50000, salaried

B age=35, salary=20000, salaried

C age=40, salary=30000, unemployed

D age=30, salary=35000, self-employed



Answer 15 — Loan Eligibility

```
# Conditions:  
# age:21-60  
# salary >= 25000  
# employment:salaried or self-employed
```

☐ A) age=19 , salary=50000 , salaried

☐ B) age=35 , salary=20000 , salaried

☐ C) age=40 , salary=30000 , unemployed

☒ D) age=30 , salary=35000 , self-employed

Explanation

Option D: age=30 (21-60 ) , salary=35000 (≥ 25000 ) , self-employed (valid type ) . All three conditions pass $\rightarrow$ Loan Approved! A fails age, B fails salary, C fails employment.

Why Other Options Are Wrong

A) age=19 — Rejected! Age must be 21-60. $19 < 21$ fails.

B) salary=20000 — Rejected! Minimum salary is 25000. $20000 < 25000$ fails.

C) unemployed — Rejected! Only 'salaried' or 'self-employed' are valid employment types.

? Day 2 — Assignment

TASK 01

? Enhanced Loan Eligibility System

Extend the Loan Eligibility System with the following rules:

- Age: 21 – 60 | Salary: $\geq$ ₹25,000 | Employment: salaried or self-employed
- If age is 21-30 AND salary $<$ ₹30,000 $\rightarrow$ 'Needs guarantor'
- If age $>$ 55 AND employment is self-employed $\rightarrow$ 'High risk, senior review needed'
- Otherwise if all conditions met $\rightarrow$ 'Approved' — else specific rejection reason

TASK 02

Smart Temperature Advisor

Build a program that reads temperature (°C) and gives activity advice:

- Below 0 $\rightarrow$ 'Freezing! Stay indoors and wear heavy clothing'
- 0 – 15 $\rightarrow$ 'Cold. A jacket is recommended'
- 16 – 25 $\rightarrow$ 'Pleasant weather! Great for outdoor activities'
- 26 – 35 $\rightarrow$ 'Hot. Stay hydrated and use sunscreen' | Above 35 $\rightarrow$ 'Extreme heat! Avoid going outside'

? Submit both programs with 3 test cases each | Deadline: Before Day 3